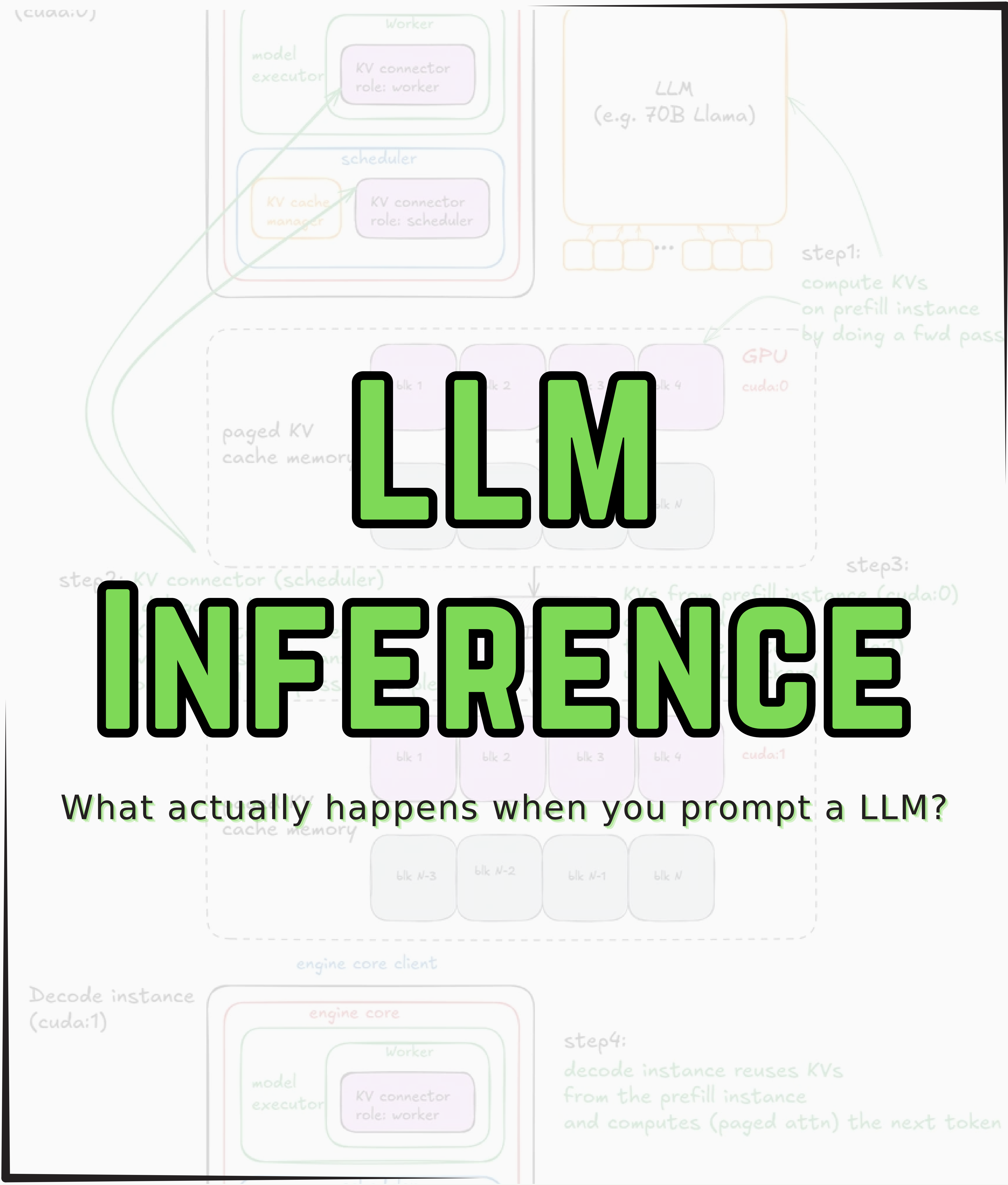


LLM Inference

What actually happens when you prompt a LLM?

Bhavishya Pandit

Introduction

Have you ever wondered what happens inside an LLM after you press ‘Enter’?

> What are integrals in mathematics?
>
> In basic terms, integrals are a way to find the total or accumulated value of something, often when you have a rate of change or something spread out over an area. For example, if you know how fast something is changing (like speed), an integral helps you figure out the total distance traveled.
>
> In geometry, integrals can help find the area under a curve on a graph. So, if you had a curve that showed how fast a car was moving at different times, an integral would help you figure out the total distance the car traveled.
>
> It's a key concept in calculus, which is a more advanced topic than what we usually cover in basic high school math, but it's a natural next step if you're interested in how rates and accumulations work together!

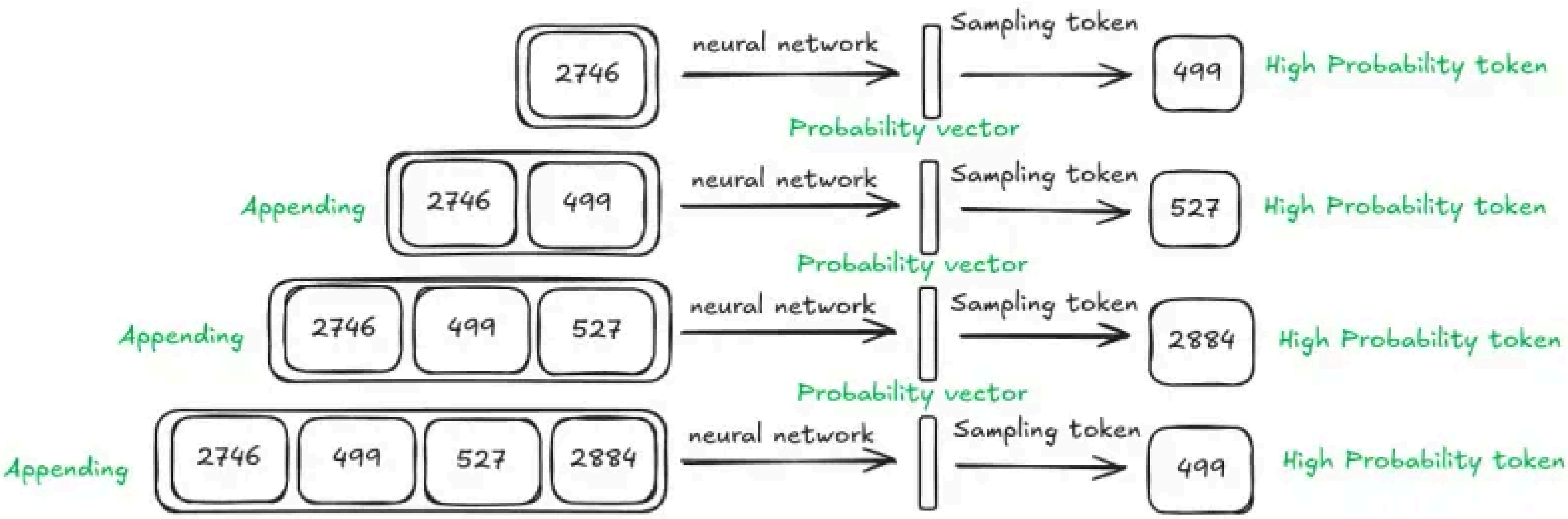
Every AI response you see involves:

- billions of parameters
- GPU-scale computation
- millisecond optimization decisions
- real monetary cost per token

In this carousel, we will break down the entire **LLM inference pipeline** that powers the modern AI systems.

What is LLM inference?

LLM inference is the process of using a pre-trained language model to generate outputs for new prompts without modifying its learned parameters.

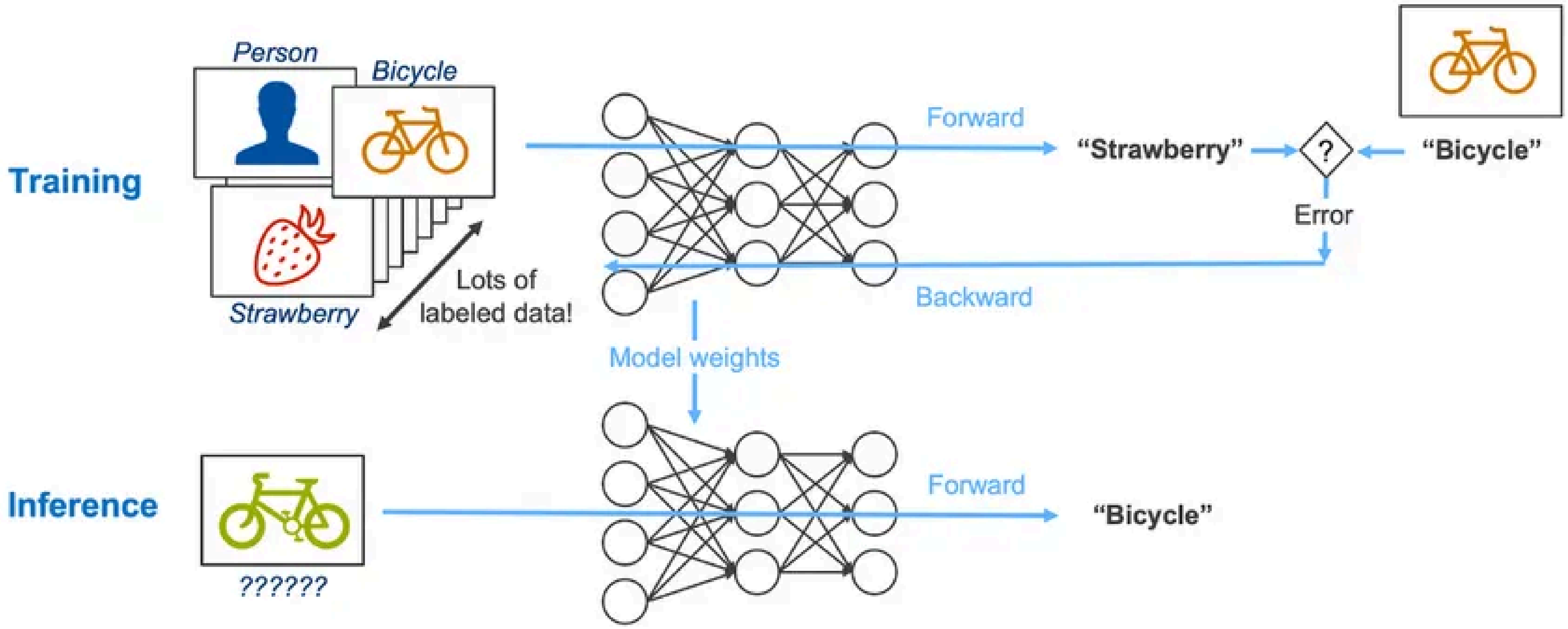


During inference, the input passes through the model's transformer layers to **compute probabilities for possible next tokens**.

And then the response is produced sequentially, one token at a time, using previously generated tokens as context (also called **autoregression**).

So, inference is the application of a trained machine learning model to new, unseen data.

Training vs Inference



During the **training phase**, the model learns language patterns.

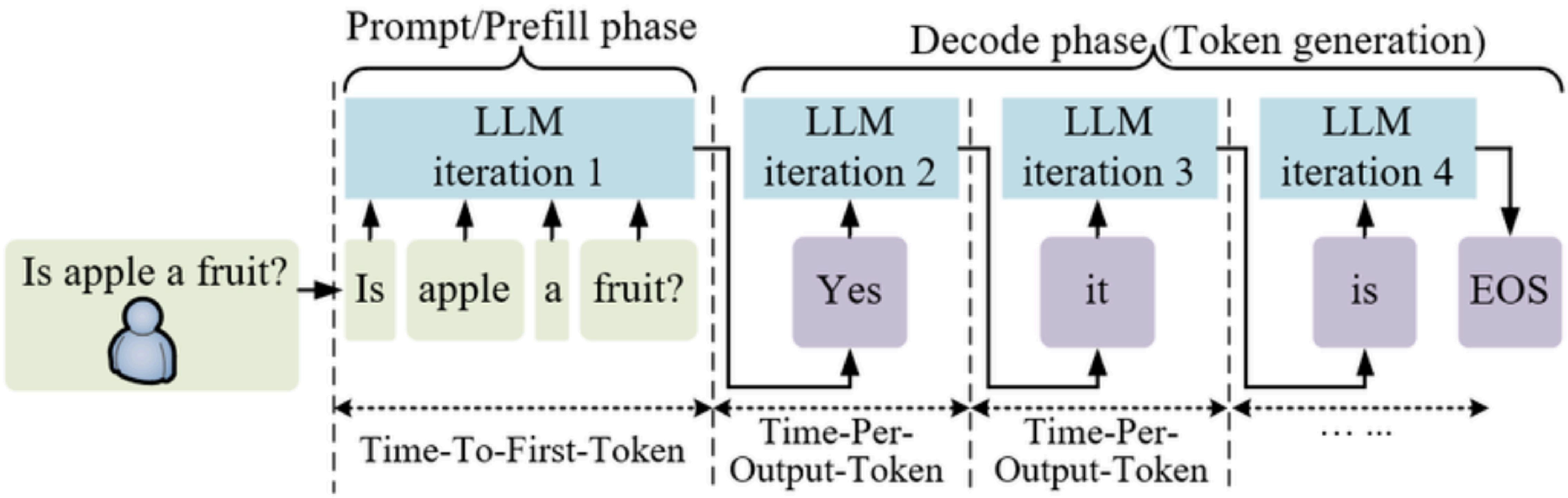
- Uses massive datasets
- Adjusts billions of parameters
- Requires weeks of GPU clusters
- Extremely expensive
- Happens rarely

While during **inference**, the model generates responses,

- No learning happens
- Parameters stay fixed
- Predicts next token repeatedly
- Runs millions of times daily
- Less expensive

How does it work?

Inference involves taking a user's input (a prompt) and processing it through the parameters of the model to generate relevant outputs like text, code, or translations.



The inference pipeline is designed for speed, accuracy and scalability and every inference request follows a four-stage pipeline:

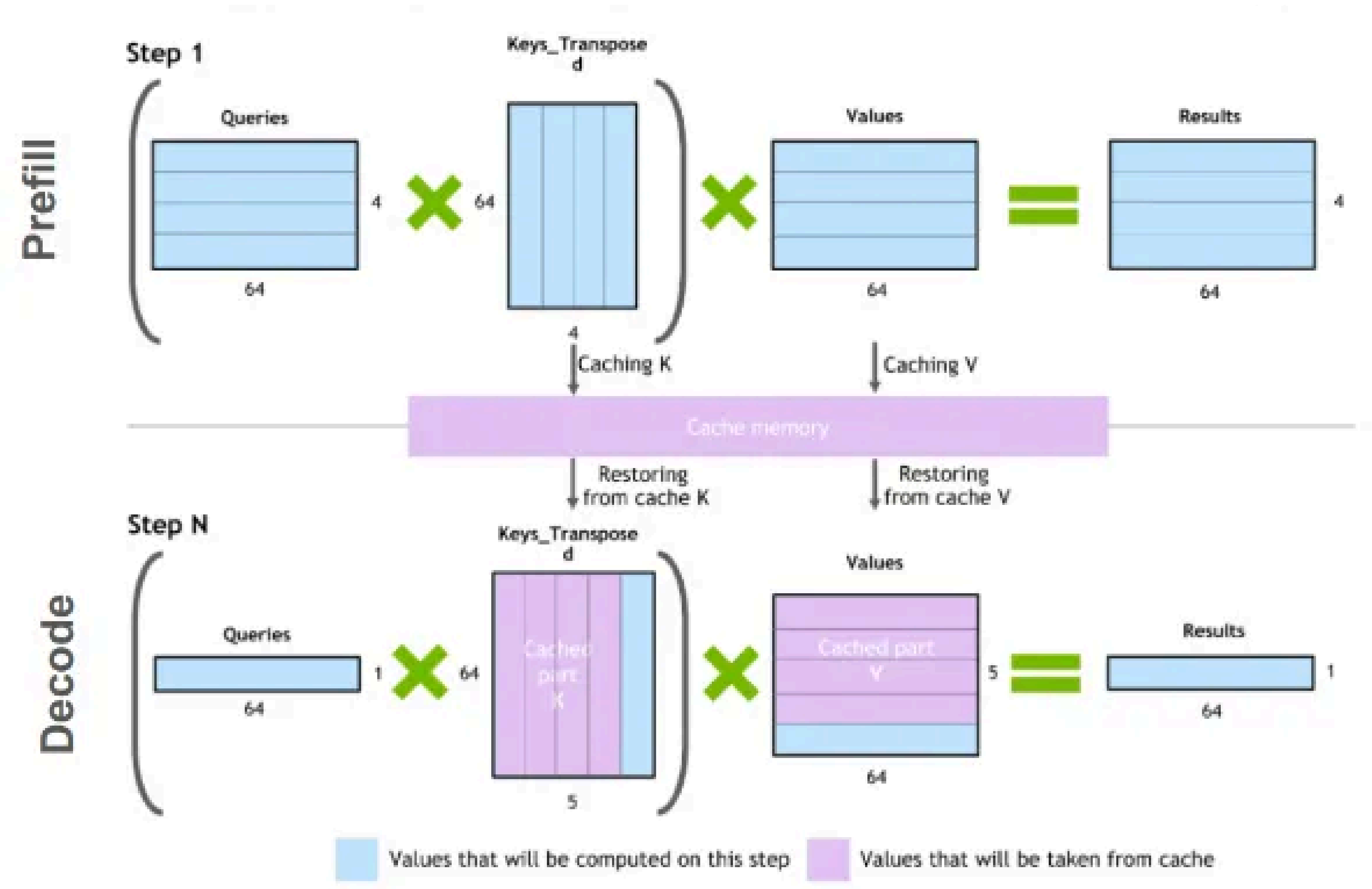
- **Tokenization** → Convert text to numerical token IDs
- **Prefill** → Process entire prompt in parallel
- **Decode** → Generate output tokens sequentially
- **Detokenization** → Convert token IDs back to text

Out of these prefill phase and decode phase are more critical. They are fundamentally different workloads and stress hardware in opposite ways.

Prefill and Decode phase

In **prefill phase**, the entire prompt is processed to build the KV cache.

- **Compute-bound**: Heavy matrix multiplications saturate GPU
- Highly parallel: All prompt tokens processed simultaneously
- Impacts TTFT: Determines Time to First Token latency



In **decode phase**, output is generated one token at a time, reading the growing KV cache.

- **Memory-bound**: Limited by bandwidth to read KV cache
- Sequential: Each token depends on previous tokens
- Impacts ITL: Determines Inter-Token Latency

Challenges

Inference powers real-world AI applications but running LLMs efficiently at scale introduces significant technical challenges.

These challenges span computational, operational, and ethical dimensions.

Performance Challenges

1. High Latency
2. Computational Cost
3. Memory Constraints

System & Infrastructure Challenges

1. Token Limits
2. Immature Tooling
3. Scalability

Reliability Challenges

1. Hallucinations
2. Accuracy

Optimization Strategies

The challenges of inference have driven rapid innovation across **models**, **hardware**, **algorithms**, and **software**.

Model Optimization

- Pruning
- Quantization
- Knowledge Distillation

Hardware Acceleration

- GPUs & TPUs
- Tensor cores
- AI accelerators

Inference Techniques

- KV Cache
- Batching
- Speculative Decoding

Software Optimization

- Dynamic batching
- Autoscaling
- Efficient runtimes

Efficient Attention Mechanisms

- Sparse Attention
- Flash Attention

Stay Ahead with Our Tech Newsletter! 🚀

👉 **Join 1.1k+ leaders and professionals to stay ahead in GenAI!**
🔗 **https://bhavishyapandit9.substack.com/**

Join our newsletter for:

- Step-by-step guides to mastering complex topics
- Industry trends & innovations delivered straight to your inbox
- Actionable tips to enhance your skills and stay competitive
- Insights on cutting-edge AI & software development

WTF In Tech

Home Notes Archive About

Object Detection with Large Vision Language Models (LVLMs)

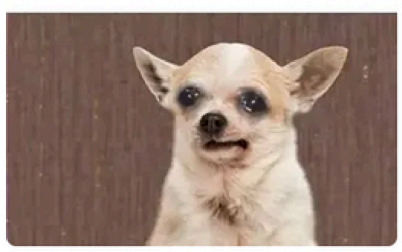
Object detection, now smarter with LVLMs

MAR 27 · BHAVISHYA PANDIT

AI Interview Playbook : Comprehensive guide to land an AI job in 2025

Brownie point: It includes 10 Key AI Interview Questions (With Answers).

MAR 22 · BHAVISHYA PANDIT



WTF In Tech

My personal Substack

💡 Whether you're a developer, researcher, or tech enthusiast, this newsletter is your shortcut to staying informed and ahead of the curve.